

What is Logistic Regression

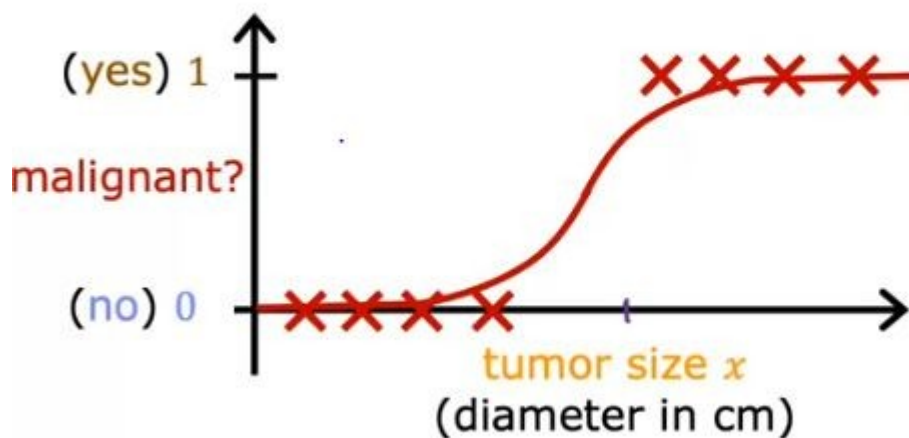
Logistic regression is a Supervised Learning Classification Algorithm. It is used to predict a binary outcome based on a set of independent variables.

A binary outcome is one where there are only two possible scenarios:

- Positive: the event happens (1)
- Negative: the event does not happen (0).

Independent variables are those variables or factors which may influence the outcome (or dependent variable).

Example: Predicting if a tumor is benign or malignant

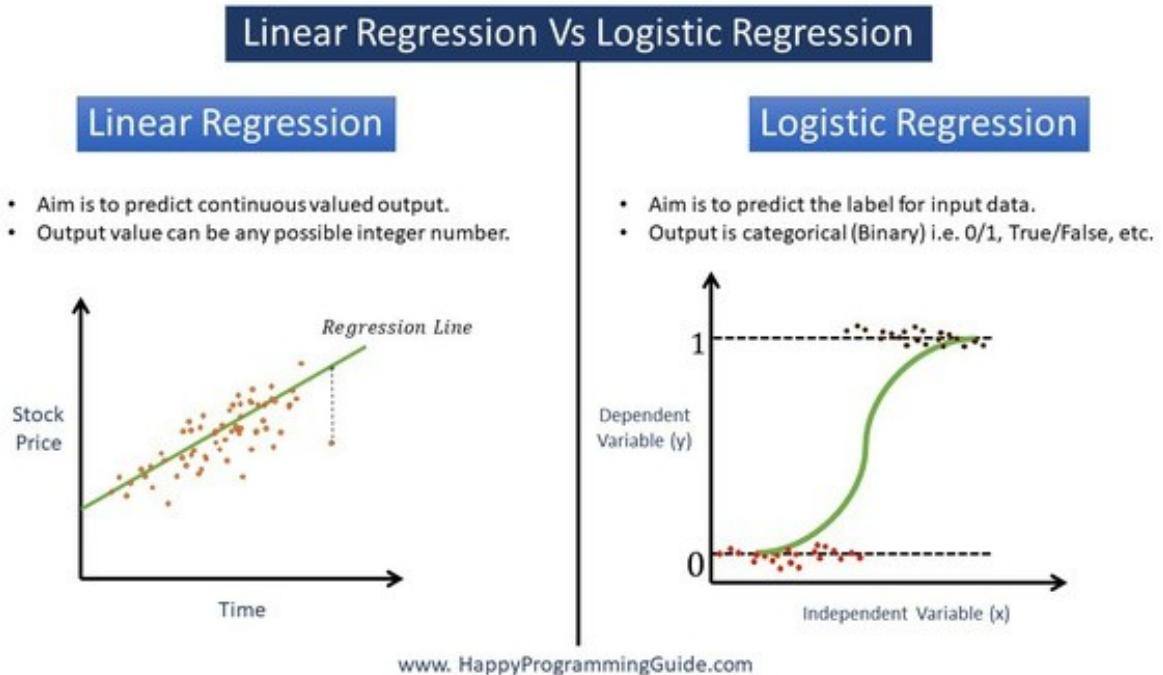


In the above example you can see that the markers at the level of $y = 1$ (yes) demonstrate examples of a malignant tumor while the examples labeled at $y = 0$ (no) demonstrate benign tumors. Logistic Regression aims to classify an input tumor marker from the user as either 1 (malignant) or 0 (benign)

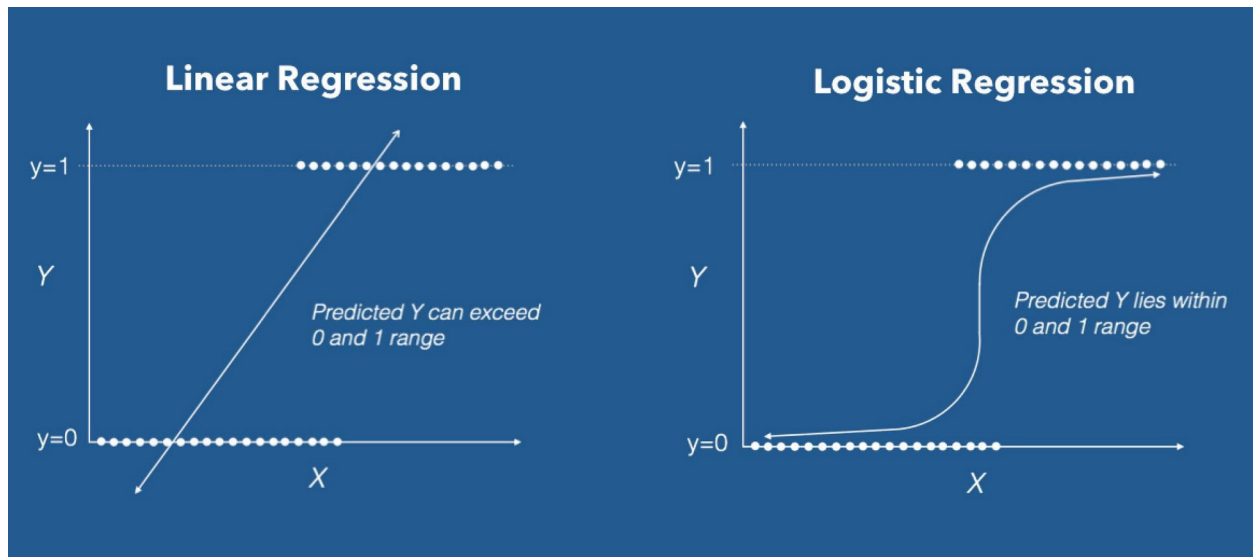
What is Linear Regression?

Linear Regression is a Supervised Learning Regression Algorithm that takes an input x and outputs a prediction y utilizing a trendline within a set of Data Points.

- Supervised learning Algorithm learns through being given a dataset which represents accurate data or "the right answers".



The difference between Linear and Logistic Regression



The Similarities with Linear Regression and Logistic Regression

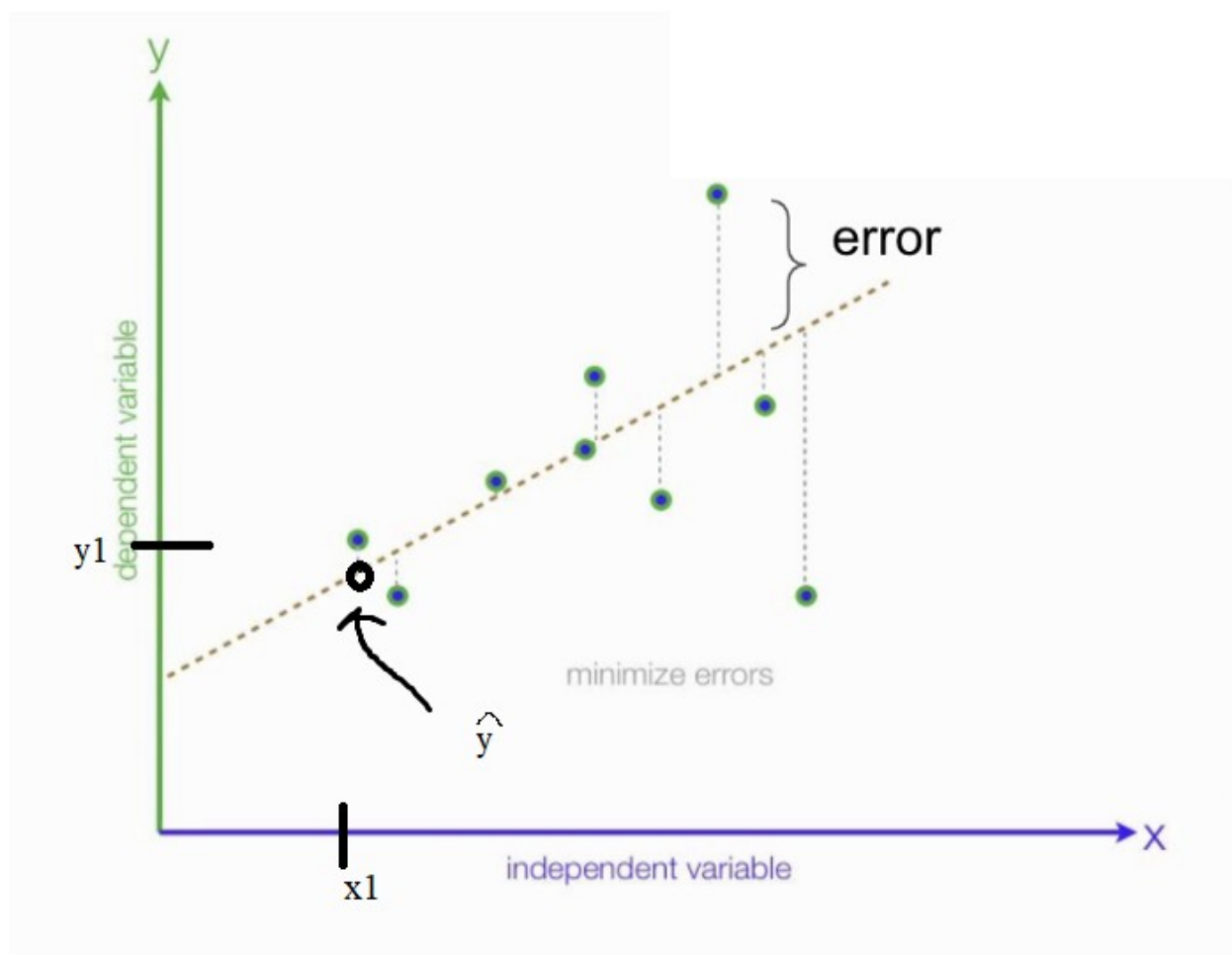
While Logistic Regression is a classification system, it shares many similarities with that of the Linear Regression Model

- Line of Best Fit/Trendline/Regression Line/ Cost Function
- Equation for Cost Function
- Implimentation of Gradient Descent

Line of Best fit

To Understand the Logistic Regression, One must first understand Linear Regression and its parameters

When conducting our linear regression there is room for error. That error falls within our line of best fit. The line of best fit has a target x_1 and y_1 . This means that for every x value there is a y value that corresponds, however where the value of x_1 meets the trendline is where the prediction is executed.



Cost : Square Error

The error that is input into the regression to accurately carry out predictions is called the Cost. The Cost is the difference between where x meets the trendline and the target variable y.

$$(\hat{y} - y)^2$$

The sum of these errors gives us the Cost Function

$$\frac{1}{2m} \sum_{i=1}^m (\underbrace{f_w(x^{(i)})}_{\hat{y}} - y^{(i)})^2$$

The Cost function represents how well our choice of trendline fits the data. This essentially helps us know the accuracy and efficacy of our regression line

Tuning Line of Best fit

The importance of the Cost Function is to find the parameters for the equation of our line of best fit. With these new parameters we would reduce our total squared error (Cost) significantly and thus have more accurate predictions.

Let's say the equation for our line is : $f_{w,b}(x) = wx + b$



We need to find the values of w and b that would make our equation fit the data with the least squared error possible



Gradient Descent

The process by which we iterate over our cost function to find the parameters of w and b that would make our function the best fit for our training data



repeat until convergence {

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

Gradient Descent Algorithm

Linear regression model

$$f_{w,b}(x) = wx + b$$

Cost function

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$$

Gradient descent algorithm

repeat until convergence {

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b) \rightarrow \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})x^{(i)}$$

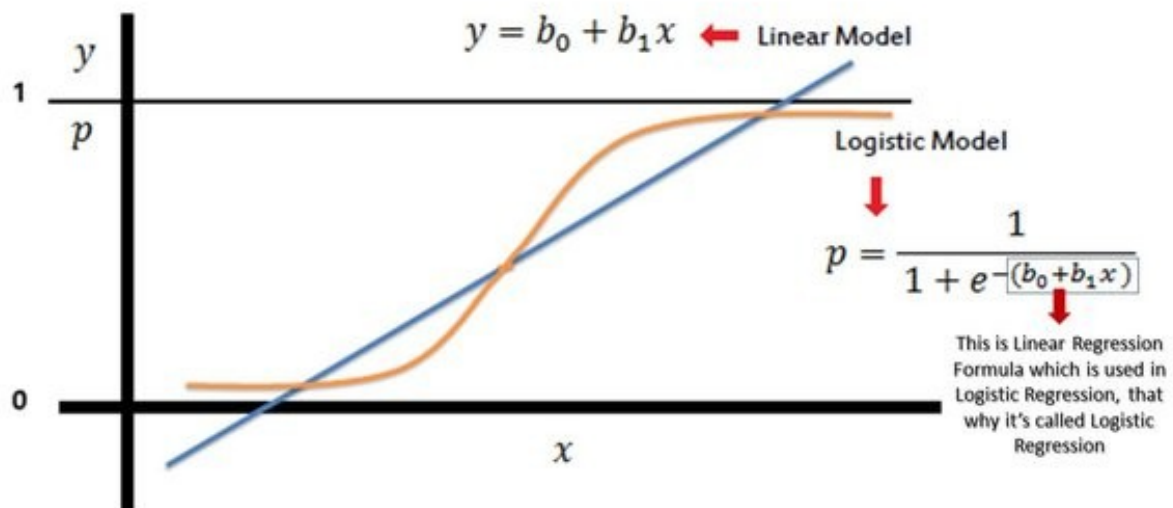
$$b = b - \alpha \frac{\partial}{\partial b} J(w, b) \rightarrow \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$$

}

α is the learning rate that helps the Gradient descent algorithm to get to its most important parameters

The above Gradient Descent Algorithm is to be run simultaneously to ensure that the Cost Function is updating with new unique parameters. This is important because any deviation will be detrimental to the Cost Function.

Cost Function : Linear Regression vs Logistic Regression



The $f(x)$ function is the major difference between calculating the cost function of the Linear Regression Model vs Logistic Regression Model

In the above function:

- Linear Regression Function : y
- Logistic Regression Function: p

The Logistic Regression model calculates the sigmoid curve of the Linear Regression function

Credit Card Fault Prediction with Logistic Regression

We will be conducting a Logistic Regression model utilizing the sklearn library. We will also run the models of Support Vector Machine and MLPClassifier to compare. The primary focus will be Logistic Regression, with a manual example of gradient descent below that.

Import Libraries and Data

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import roc_curve, auc

data =
pd.read_csv('../input/default-of-credit-card-clients-dataset/UCI_Credit_Card.csv')
data
```

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2
PAY_3 \								
0	1	20000.0	2	2	1	24	2	2
-1								
1	2	120000.0	2	2	2	26	-1	2
0								
2	3	90000.0	2	2	2	34	0	0
0								
3	4	50000.0	2	2	1	37	0	0
0								
4	5	50000.0	1	2	1	57	-1	0
-1								
...	...	...	...	...	...	...	...	...
...								
29995	29996	220000.0	1	3	1	39	0	0
0								
29996	29997	150000.0	1	3	2	43	-1	-1
-1								
29997	29998	30000.0	1	2	2	37	4	3

2								
29998	29999	80000.0	1	3	1	41	1	-1
0								
29999	30000	50000.0	1	2	1	46	0	0
0								
	PAY_4	...	BILL_AMT4	BILL_AMT5	BILL_AMT6	PAY_AMT1	PAY_AMT2	
\								
0	-1	...	0.0	0.0	0.0	0.0	689.0	
1	0	...	3272.0	3455.0	3261.0	0.0	1000.0	
2	0	...	14331.0	14948.0	15549.0	1518.0	1500.0	
3	0	...	28314.0	28959.0	29547.0	2000.0	2019.0	
4	0	...	20940.0	19146.0	19131.0	2000.0	36681.0	
...	...	...	...	...	...	...	...	
29995	0	...	88004.0	31237.0	15980.0	8500.0	20000.0	
29996	-1	...	8979.0	5190.0	0.0	1837.0	3526.0	
29997	-1	...	20878.0	20582.0	19357.0	0.0	0.0	
29998	0	...	52774.0	11855.0	48944.0	85900.0	3409.0	
29999	0	...	36535.0	32428.0	15313.0	2078.0	1800.0	
	PAY_AMT3	PAY_AMT4	PAY_AMT5	PAY_AMT6				
default.payment.next.month								
0	0.0	0.0	0.0	0.0				
1								
1	1000.0	1000.0	0.0	2000.0				
1								
2	1000.0	1000.0	1000.0	5000.0				
0								
3	1200.0	1100.0	1069.0	1000.0				
0								
4	10000.0	9000.0	689.0	679.0				
0								
...	...	...	...	...				
...								
29995	5003.0	3047.0	5000.0	1000.0				
0								
29996	8998.0	129.0	0.0	0.0				
0								
29997	22000.0	4200.0	2000.0	3100.0				

```

1
29998      1178.0      1926.0      52964.0      1804.0
1
29999      1430.0      1000.0      1000.0      1000.0
1

```

[30000 rows x 25 columns]

```
data.info()
```

*# There are no Object columns or NA columns thus we can move on to
Cleansing the Data*

*#It also helps that these values are all numerical as we can issue a
correlation heatmap*

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 30000 entries, 0 to 29999
```

```
Data columns (total 25 columns):
```

#	Column	Non-Null Count	Dtype
0	ID	30000 non-null	int64
1	LIMIT_BAL	30000 non-null	float64
2	SEX	30000 non-null	int64
3	EDUCATION	30000 non-null	int64
4	MARRIAGE	30000 non-null	int64
5	AGE	30000 non-null	int64
6	PAY_0	30000 non-null	int64
7	PAY_2	30000 non-null	int64
8	PAY_3	30000 non-null	int64
9	PAY_4	30000 non-null	int64
10	PAY_5	30000 non-null	int64
11	PAY_6	30000 non-null	int64
12	BILL_AMT1	30000 non-null	float64
13	BILL_AMT2	30000 non-null	float64
14	BILL_AMT3	30000 non-null	float64
15	BILL_AMT4	30000 non-null	float64
16	BILL_AMT5	30000 non-null	float64
17	BILL_AMT6	30000 non-null	float64
18	PAY_AMT1	30000 non-null	float64
19	PAY_AMT2	30000 non-null	float64
20	PAY_AMT3	30000 non-null	float64
21	PAY_AMT4	30000 non-null	float64
22	PAY_AMT5	30000 non-null	float64
23	PAY_AMT6	30000 non-null	float64
24	default.payment.next.month	30000 non-null	int64

```
dtypes: float64(13), int64(12)
```

```
memory usage: 5.7 MB
```

Visualization

A large part of Regression Models has to do with correlations. Correlations help the algorithms to detect what features influence each other or the desired outcomes. Heatmaps are a very impactful way to visualize these correlations

```
#Find the correlation of the dataset
```

```
corr= data.corr()
```

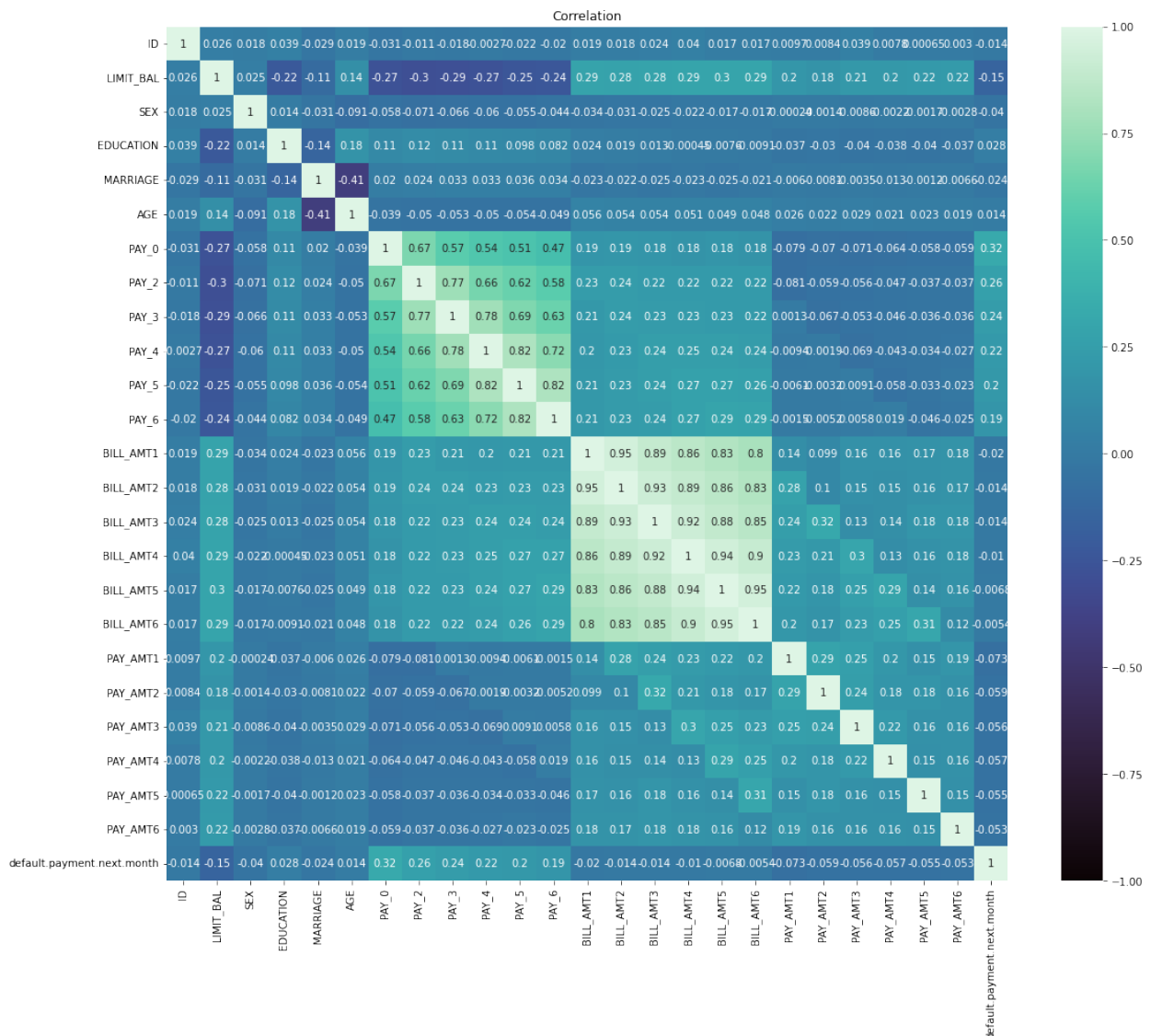
```
#Plot heatmap
```

```
plt.figure(figsize=(18,15))
```

```
sns.heatmap(corr, annot=True, vmin=-1.0, cmap = 'mako')
```

```
plt.title('Correlation')
```

```
plt.show()
```



Cleanse Data

An important factor to our regression model is the type of our features.

A feature could be:

- Nominal:

Has no difference in magnitude dictated by the columns, ex:sex, marriage. We don't want our algorithm to think that there is an ordered difference of sex or education. This means if 1 denotes male and 2 denotes female we don't want our model to think of this as an actual numerical difference. Thus we would choose to separate the features

- Ordinal:

Used to demonstrate the order or rank of elements in a collection

Therefore, Nominal = identity, Ordinal = magnitude

One hot encoding

One hot encoding is one method of converting data to prepare it for an algorithm and get a better prediction. With one-hot, we convert each categorical value into a new categorical column and assign a binary value of 1 or 0 to those columns. Each integer value is represented as a binary vector.

```
#Setup dummies to split nominal features into separate columns
def onehotencode(df, columndict):
    df= df.copy()

    for column, prefix in columndict.items():
        dummies= pd.get_dummies(df[column], prefix = prefix)
        #concat to original dataframe
        df = pd.concat([df, dummies], axis=1)
        df = df.drop(column, axis=1)

    return df

# Cleanse Data

def cleaner(df):
    df = df.copy()

    #We do not need the ID column index
    df=df.drop('ID', axis=1)

    df=onehotencode(
        df,
        {
```

```

        'EDUCATION': 'EDU',
        'MARRIAGE': 'MAR'
    }
)

#Turn Data Set into X and y sets
#Use the copy function so we can keep our original data frames at
all times
y = df['default.payment.next.month'].copy()
X = df.drop('default.payment.next.month', axis=1).copy()

#Scale our Data
scaler = StandardScaler()
X = pd.DataFrame(scaler.fit_transform(X), columns = X.columns)

return X, y

```

X,y = cleaner(data)

X

	LIMIT_BAL	SEX	AGE	PAY_0	PAY_2	PAY_3
PAY_4 \						
0	-1.136720	0.810161	-1.246020	1.794564	1.782348	-0.696663 -
0.666599						
1	-0.365981	0.810161	-1.029047	-0.874991	1.782348	0.138865
0.188746						
2	-0.597202	0.810161	-0.161156	0.014861	0.111736	0.138865
0.188746						
3	-0.905498	0.810161	0.164303	0.014861	0.111736	0.138865
0.188746						
4	-0.905498	-1.234323	2.334029	-0.874991	0.111736	-0.696663
0.188746						
...	...	...	...	...	...	...
...						
29995	0.404759	-1.234323	0.381275	0.014861	0.111736	0.138865
0.188746						
29996	-0.134759	-1.234323	0.815221	-0.874991	-0.723570	-0.696663 -
0.666599						
29997	-1.059646	-1.234323	0.164303	3.574267	2.617654	1.809921 -
0.666599						
29998	-0.674276	-1.234323	0.598248	0.904712	-0.723570	0.138865
0.188746						
29999	-0.905498	-1.234323	1.140680	0.014861	0.111736	0.138865
0.188746						
	PAY_5	PAY_6	BILL_AMT1	...	EDU_1	EDU_2

EDU_3 \							
0	-1.530046	-1.486041	-0.642501	...	-0.738375	1.066900	-
0.442752							
1	0.234917	1.992316	-0.659219	...	-0.738375	1.066900	-
0.442752							
2	0.234917	0.253137	-0.298560	...	-0.738375	1.066900	-
0.442752							
3	0.234917	0.253137	-0.057491	...	-0.738375	1.066900	-
0.442752							
4	0.234917	0.253137	-0.578618	...	-0.738375	1.066900	-
0.442752							
...	...	...	...	...	...	...	..
.							
29995	0.234917	0.253137	1.870379	...	-0.738375	-0.937295	
2.258602							
29996	0.234917	0.253137	-0.672786	...	-0.738375	-0.937295	
2.258602							
29997	0.234917	0.253137	-0.647227	...	-0.738375	1.066900	-
0.442752							
29998	0.234917	-0.616452	-0.717982	...	-0.738375	-0.937295	
2.258602							
29999	0.234917	0.253137	-0.044739	...	-0.738375	1.066900	-
0.442752							

	EDU_4	EDU_5	EDU_6	MAR_0	MAR_1	MAR_2	
MAR_3							
0	-0.064163	-0.097063	-0.041266	-0.042465	1.093780	-1.066471	-
0.104326							
1	-0.064163	-0.097063	-0.041266	-0.042465	-0.914261	0.937672	-
0.104326							
2	-0.064163	-0.097063	-0.041266	-0.042465	-0.914261	0.937672	-
0.104326							
3	-0.064163	-0.097063	-0.041266	-0.042465	1.093780	-1.066471	-
0.104326							
4	-0.064163	-0.097063	-0.041266	-0.042465	1.093780	-1.066471	-
0.104326							
...	...	...	...	...	...	...	
...							
29995	-0.064163	-0.097063	-0.041266	-0.042465	1.093780	-1.066471	-
0.104326							
29996	-0.064163	-0.097063	-0.041266	-0.042465	-0.914261	0.937672	-
0.104326							
29997	-0.064163	-0.097063	-0.041266	-0.042465	-0.914261	0.937672	-
0.104326							
29998	-0.064163	-0.097063	-0.041266	-0.042465	1.093780	-1.066471	-
0.104326							
29999	-0.064163	-0.097063	-0.041266	-0.042465	1.093780	-1.066471	-
0.104326							

```
[30000 rows x 32 columns]
```

Train the Models

You want to train your models against others to see their performance and what models could be done better

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
train_size=0.7, random_state=1234) #Set state to be able to reproduce values automatically
```

```
models = {
    LogisticRegression(): "Logistic Regression",
    SVC(): "Support Vector Machine",
    MLPClassifier(): "Neural Network"
}
```

```
for model in models.keys():
    model.fit(X_train, y_train)
```

```
/opt/conda/lib/python3.7/site-packages/sklearn/neural_network/_multilayer_perceptron.py:696: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  ConvergenceWarning,
```

```
for model, name in models.items():
    print(name+ " : {:.2f}%".format(model.score(X_test, y_test)*100))
```

```
Logistic Regression: 81.14%
Support Vector Machine: 82.12%
Neural Network: 80.81%
```

As seen above Support Vector Machine was the most accurate in predicting the defaults on credit card clients. However as this is a Logistic Regression Model we will now go into the actual evaluation of this dataset.

ROC and AUC

The Receiver Operating Characteristics(ROC) is a measure of a classification model's performance over various thresholds. This is accomplished using the parameters of a Confusion Matrix. The Area Under the Curve is a metric to evaluate the ROC of various classification models.

ROC measures model's accuracy

AUC measures model's accuracy vs other models

Confusion Matrix

		Actual Value	
		Positive [have disease]	Negative [no disease]
Predicted Value	Positive [have disease]	TP	FP
	Negative [no disease]	FN	TN

$specificity = \frac{TN}{FP + TN}$ $fall-out = \frac{FP}{FP + TN}$

Compare SVM and Logistic regression for ROC and AUC

```
model_SVC = SVC(kernel = 'rbf', random_state = 1234)
model_SVC.fit(X_train, y_train)
y_pred_svm = model_SVC.decision_function(X_test)

model_logistic = LogisticRegression()
model_logistic.fit(X_train, y_train)
y_pred_logistic = model_logistic.decision_function(X_test)

#Plot ROC and compare AUC
logistic_fpr, logistic_tpr, threshold = roc_curve(y_test,
y_pred_logistic)
auc_logistic = auc(logistic_fpr, logistic_tpr)

svm_fpr, svm_tpr, threshold = roc_curve(y_test, y_pred_svm)
auc_svm = auc(svm_fpr, svm_tpr)
```

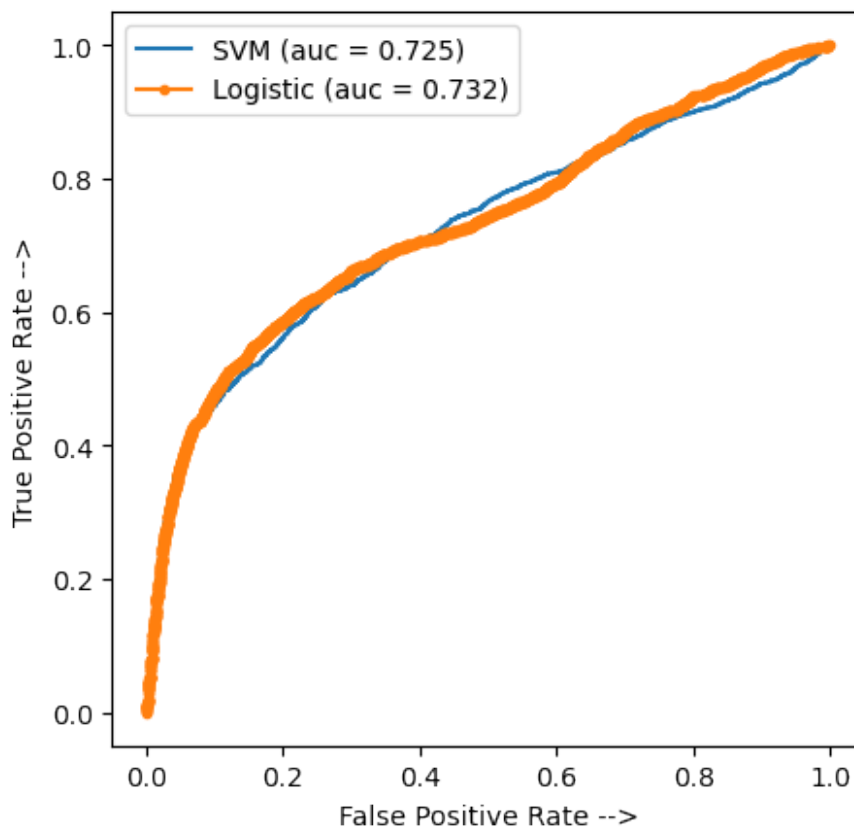


```
plt.figure(figsize=(5, 5), dpi=100)
plt.plot(svm_fpr, svm_tpr, linestyle='--', label='SVM (auc = %0.3f)' %
auc_svm)
plt.plot(logistic_fpr, logistic_tpr, marker='.', label='Logistic (auc
= %0.3f)' % auc_logistic)

plt.xlabel('False Positive Rate -->')
plt.ylabel('True Positive Rate -->')

plt.legend()

plt.show()
```



Above we see that in this score the Logistic Regression has the larger AUC and could be seen as a stronger model regarding this metric.

Conclusion

The Logistic Regression is an amazing tool that aids in the creation of classification. There are various means cases for utilization for the Logistic Regression model as there are various evaluation metrics to judge model performance. I hope this tutorial helped.